Facens

Disciplina

Algoritmos e Programação

Conteúdo

Funções

Ao final desta aula o estudante deverá

Ser capaz de desenvolver algoritmos que utilizem funções em Linguagem C

Facens
AQUI TEM ENGENHARIA
material didático

Conceito

- **Função** é um bloco de códigos, sejam declarações e outros comandos, que podem ser nomeados e chamados dentro da função principal do programa.
- Uma função também é chamada de subprograma.
- A linguagem C possui muitas funções já implementadas e agrupadas nas bibliotecas.
- Exemplo: `main()`, `printf()`, `scanf()`, `getch()`, `toupper()`, `sqrt()`, `pow()`.

Facens
AQUI TEM ENGENHARIA
material didático

Conceito

- **Função definida pelo usuário** é uma seção de código independente e autônoma, escrita para desempenhar uma tarefa específica. Deve conter protótipo e definição da função.
- Uma **função** pode receber informações através de argumentos, cada função tem um nome único e suas instruções são executadas sempre que esse nome é mencionado no programa, isto é conhecido como "chamar uma função".

Facens
AQUI TEM ENGENHARIA
material didático

Conceito

- As principais razões para o uso de funções são:
  1. **Estruturação dos programas**: onde o programa é estruturado através de funções que possuem tarefas específicas e bem definidas.
  2. **Reutilização de código:** permite que o bloco de comandos que compõem a função pode ser utilizado diversas vezes e em pontos diferentes do programa.

Facens
AQUI TEM ENGENHARIA
material didático

Protótipo da Função

- Fornece ao compilador a descrição de uma função que será definida posteriormente no programa.
- Sintaxe:

```
tipo_retorno nome_função (tipo_param nome,...);
```

  - **tipo_retorno** – tipo de dado que a função retornará. Pode ser char, int, float, double e void (se não retornar nada).
  - **nome_função** – descreve o que a função faz.
  - **tipo_param nome** – tipo e nome das variáveis que serão passados para a função. Pode ser char, int, float, double e void.

Facens
AQUI TEM ENGENHARIA
material didático

Protótipo da Função

- Exemplos de protótipos:
  - `int produto(int x, int y);`
  - `float metade(int k);`
  - `void moldura( );`
  - `int seleciona(void);`
  - `void func1(int x, int y, int z);`

Facens
AQUI TEM ENGENHARIA
material didático

Definição da Função

- Contém as instruções que serão executadas. É composta pelo cabeçalho e pelo corpo da função.
- Sintaxe:

```
tipo_retorno nome__função (tipo_param nome,...)//cabeçalho
{
  instruções; //corpo da função
}
```

  - **tipo_retorno** – tipo de dado que a função retornará. Pode ser char, int, float, double e void (se não retornar nada).
  - **nome_função** – descreve o que a função faz.
  - **tipo_param nome** – tipo e nome das variáveis que serão passados para a função. Pode ser char, int, float, double, void.

Facens
AQUI TEM ENGENHARIA
material didático

Definição da Função

- Exemplo de definição da função:

```
void moldura( )                    // cabeçalho
{                                  // abertura de chaves
     printf("********\n");
     printf("*      *\n");         // corpo da função
     printf("********");
}                                  // fechamento de chaves
```


Facens
AQUI TEM ENGENHARIA
material didático

Chamada da Função

- A chamada da função é a referência da função no programa. Ao realizar a chamada executa-se a função.
- A chamada ocorre através do nome da função, seguido de parênteses, podendo conter ou não a lista de argumentos.
- Exemplo:
  - `produto(9, 654); // nome, seguido de parênteses contendo lista de argumentos`
  - `metade(x);`
  - `moldura( ); // nome seguido de parênteses`
  - `seleciona( );`
  - `func1(a, b, c);`

Facens
AQUI TEM ENGENHARIA
material didático

Chamada da Função

- O mecanismo utilizado para transmitir informação para uma função é chamado argumento.
- Os argumentos podem ser constantes, variáveis, expressões matemáticas, expressões lógicas e até outra função.
- Exemplo:

```
b = quadrado (metade (y));        /* a = metade (y);
                                     b = quadrado (a) */
```


Facens
AQUI TEM ENGENHARIA
material didático

Chamada da Função

- Os argumentos devem ser compatíveis com os parâmetros em quantidade e tipo de dado.
- Exemplo:
  - Chamada: `func1( a, b, c );` argumentos a, b e c
  - Cabeçalho: `void func1(int x,int y,int z)` parâmetros x, y e z
- Argumentos: são valores reais.
- Parâmetros: são "guardadores de lugar" para um argumento, são fixos, contém o tipo e o nome dos dados.

Facens
AQUI TEM ENGENHARIA
material didático

Tipos de Funções

- Há funções criadas pelo programador que não retornam nada para o programa e outras que retornam valores.
- **Funções que não retornam valores**: são aquelas declaradas como void e que no corpo da função contém apenas instruções e comandos.
- Exemplo:

Protótipo: `void moldura( );`

Chamada: `moldura( );`

Definição da função:

```
void moldura( )
{
   printf("********\n");
   printf("*      *\n");
   printf("********");
 }
```


Facens
AQUI TEM ENGENHARIA
material didático

Tipos de Funções

- **Funções que retornam valores:** são aquelas declaradas como char, int, float ou double. O valor de retorno deve ser armazenado em uma variável ou passado como argumento. No corpo da função além das instruções e comando há necessariamente o comando **return( )**.
- Exemplo:

Protótipo: `float divisao( int x, int y );`

Chamada: `d=divisao(a,b)` ou `printf("%f", divisao(a,b));`

Definição da função:

```
float divisao(int x,int y)
{
    return (x/y);
}
```

ou

```
float divisao(int x,int y)
{
    float resp;
    resp=x/y;
    return resp;
}
```


Facens
AQUI TEM ENGENHARIA
material didático

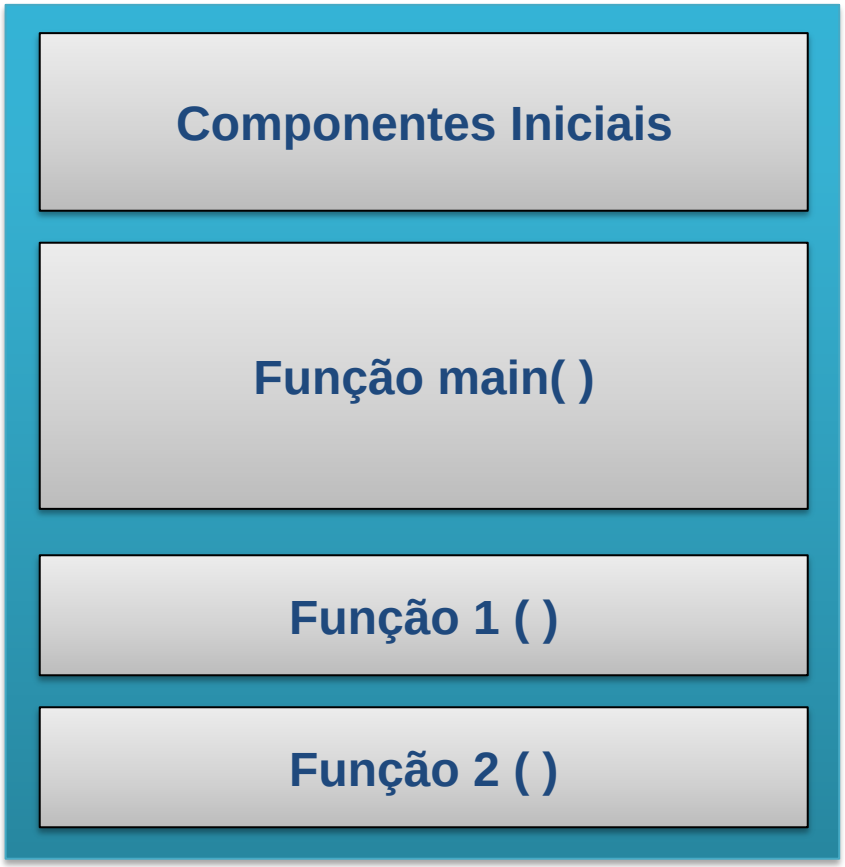
Tipos de Funções

- O comando **return** pode estar acompanhado ou não:
  - Return (expressão) ou return variável – comando que devolve um único valor e retorna, imediatamente, para a próxima instrução do código de chamada.
  - return – sem qualquer valor ou variável, causa uma saída imediata da função na qual ela se encontra, porém sem retornar qualquer valor.

- Uma função encerra sua execução quando:
  - o fim do seu código é atingido ou
  - um comando return é encontrado e executado.

Facens
AQUI TEM ENGENHARIA
material didático

Estrutura do programa modular



- Contém: arquivos de cabeçalho, declaração de constantes, protótipos e variáveis globais
- Contém: variáveis locais e instruções do programa
- Contém: variáveis locais e definição da função 1
- Contém: variáveis locais e definição da função 2

Facens
AQUI TEM ENGENHARIA
material didático

Funções – Chamada por Valor

- **Ex1:** Receber uma temperatura em Fahrenheit e convertê-la para Celsius. Exibir o valor em Celsius. `c = (f-32)/1.8`

- **Ex2:** Crie um programa que calcula a área de uma esfera através de uma função. $A = 4 * \pi * r^2$

- **Ex3:** Calcular o consumo médio (Cm) de gasolina de um tanque de automóvel. Pedir para o usuário entrar com a distância (km) e volume (litros). Utilize a seguinte fórmula na função de consumo médio: `Cm = d (km) / v (litros).`

Facens
AQUI TEM ENGENHARIA
material didático

Funções – Chamada por Valor

- **Ex4:** Escreva um programa com uma função que receba um número de entrada e retorne 1 se o número lido for par e 0 se não for par. Na função main() mostre a mensagem apropriada para cada caso.
- **Ex5:** Desenvolver um programa com uma função que determina e retorna a potência de um número. A função deverá receber os parâmetros de base e expoente da potência, com um número positivo, e sem utilizar funções prontas deverá realizar o cálculo através de um processo de repetição.
    - **Por exemplo**: para as entradas base= 3 e expoente = 4, a função deverá retornar 81 fazendo, 3^4 = 3 x 3 x 3 x 3 = 81.

Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exemplo 1:**
  - Receber uma temperatura em Fahrenheit e convertê-la para Celsius. Exibir o valor em Celsius.

```
#include<stdio.h>
#include<conio.h>
main()
{
    float c,f;
    printf("Digite o valor da temperatura (Fahrenheit) :");
    scanf("%f",&f);

    c =(f-32)/1.8;

    printf("\nA temperatura \x82 igual a %.3f\xF8 C. ",c);

    getch();
    return 0;
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

```
#include<stdio.h>
#include<conio.h>
float converte (float fah);//Protótipo

main()
{
   float c,f;
   printf("Digite o valor da temperatura (Fahrenheit) :");
   scanf("%f",&f);
   c =converte(f); //Chamada da função
   printf("\nA temperatura \x82 igual a %.3f\xF8 C. ",c);
   getch();
   return 0;
}
float converte (float fah) //Definição da função
{
    float celsius;
    celsius=(fah-32)/1.8;
    return celsius;
}
```

Chamada por valor

Facens
AQUI TEM ENGENHARIA
material didático

Funções

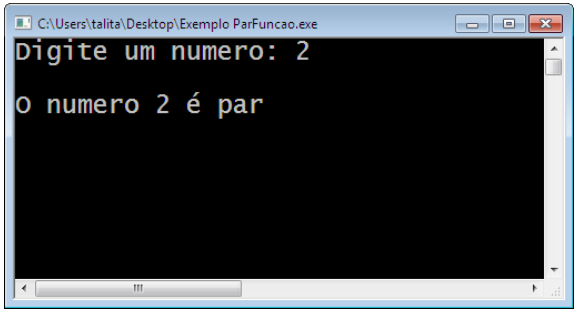
- **Exemplo 2:**
  - Escreva um programa com uma função que receba um número de entrada e retorne 1 se o número lido for par e 0 se não for par. Na função main() mostre a mensagem apropriada para cada caso.

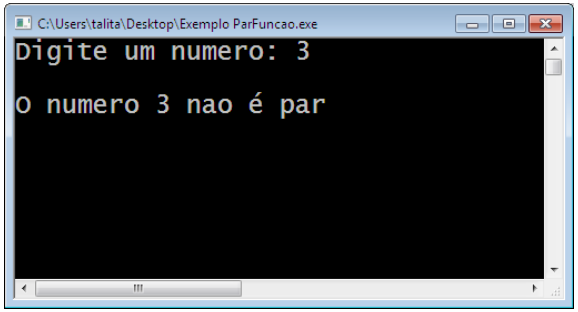
Facens
AQUI TEM ENGENHARIA
material didático

Funções

```
#include<stdio.h>

int par (int numero);

int main()
{
    int num,verifica;

    printf("Digite um numero: ");
    scanf("%i",&num);

    verifica=par(num);
    if (verifica == 1)
    {
        printf("\nO numero %i \x82 par",num);
    }
    else
    {
        printf("\nO numero %i nao \x82 par",num);
    }
    getch();
    return 0;
}

int par(int numero)
{
    if (numero % 2 == 0)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}
```

Chamada por valor





Facens
AQUI TEM ENGENHARIA
material didático

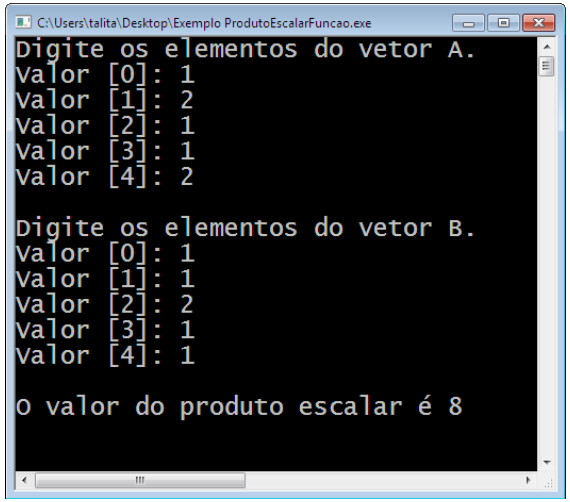
Funções

- **Exemplo 3:**
  - Escreva um programa com uma função que receba dois vetores de entrada e retorne o produto escalar. Na função main() mostre o valor do produto escalar.

Facens
AQUI TEM ENGENHARIA
material didático

Funções

```
#include<stdio.h>
#define N 5
int produtoEscalar(int vetorA[], int vetorB[]);
int main()
{
    int A[N],B[N],i;

    printf("Digite os elementos do vetor A.\n");
    for(i=0;i<N;i++)
    {
        printf("Valor [%i]: ", i);
        scanf("%i",&A[i]);
    }
    printf("\nDigite os elementos do vetor B.\n");
    for(i=0;i<N;i++)
    {
        printf("Valor [%i]: ", i);
        scanf("%i",&B[i]);
    }
    printf("\nO valor do produto escalar \x82 %i",produtoEscalar(A,B));
    getch();
    return 0;
}

int produtoEscalar(int vetorA[], int vetorB[])
{
    int i,pe=0;
    for(i=0;i<N;i++)
    {
        pe=pe+vetorA[i]*vetorB[i];
    }
    return pe;
}
```



Chamada por referência

Para **matriz** o primeiro par de colchetes pode sempre ser vazio e os demais preenchidos.
Ex.:
int verificaLinha(int velha[ ][3])

Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 1:**
  - Ler 2 notas e calcular a média. Crie uma função que receberá como parâmetro de entrada as duas notas e retorna a média calculada.

$$\boxed{\overline{X} = \frac{\sum x}{n}}$$

Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 1:**

```
#include<stdio.h>
float media (float a, float b); //Protótipo
main()
{       float n1, n2;
        printf("Digite dois valores: ");
        scanf("%f %f", &n1,&n2);
        //Chamada da função
        printf("\nA media \x82 igual a %.2f.",media(n1,n2));
        getch();
        return 0;
}
float media (float a, float b) //Definição da função
{
        float m;
        m=(a+b)/2;
        return m;
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 1: Usando 2 funções**

```
#include<stdio.h>
float media (float a, float b); //Protótipo Media
float soma (float a, float b);  //Protótipo Soma
int main()
{   float n1, n2;
    printf("Digite dois valores: ");
    scanf("%f %f", &n1,&n2);
    //Chamada da função
    printf("\nA media \x82 igual a %.2f.",media(n1,n2));
    getch();
    return 0;
}
float media (float a, float b) //Definição da função
{
    return (soma(a,b)/2);
}
float soma (float a, float b) //Definição da função
{
    return (a+b);
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 2:**
  - Desenvolver um programa com uma função que determina e retorna a potência de um número. A função deverá receber os parâmetros de base e expoente da potência, com um número positivo, e sem utilizar funções prontas deverá realizar o cálculo através de um processo de repetição.
    - **Por exemplo**: para as entradas base= 3 e expoente = 4, a função deverá retornar 81 fazendo, 3^4 = 3 x 3 x 3 x 3 = 81.

$$\boxed{x^n}$$

Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 2:**

```
#include<stdio.h>
float pot (float base,float expoente);//Protótipo
main()
{
    float b,e;
    do
    {
         printf("Digite base e expoente (b^e):");
         scanf("%f %f",&b,&e);
    }while(e<0);
    //Chamada da função
    printf("\nO valor calculado \x82 igual a %.2f.", pot(b,e));
    getch();
    return 0;
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 2:**

```
//Definição da função
float pot (float base,float expoente)
{
    float resultado=1;
    int i;
    for(i=1;i<=expoente;i++)
        resultado*=base;
    return resultado;
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 3:**
  - Faça um programa que leia um vetor de 10 posições de números reais e chame uma função para encontrar e retornar o maior valor do vetor, a seguir divida todos os elementos do vetor pelo maior valor encontrado. Mostre o vetor após os cálculos.

Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 3:**

```
#include<stdio.h>
void recebe_vetor(float v[]);
float acha_maior(float v[]);
void divide_maior(float v[], float m);
void imprime_vetor(float v[]);
main()
{       float vetor[10],maior;
        recebe_vetor(vetor);
        printf("\nVetor Original:\n");
        imprime_vetor(vetor);
        maior = acha_maior(vetor);
        printf("\nMaior = %.2f\n\n",maior);
        divide_maior(vetor, maior);
        printf("\nVetor Modificado:\n");
        imprime_vetor(vetor);
        getch();
        return 0;
}
```

Funções

- **Exercício 3:**

```
void recebe_vetor(float v[])
{
        int i;
        for(i=0;i<10;i++)
        {
            printf("Digite o elemento [%i]: ",i);
            scanf("%f", &v[i]);
        }
}
void imprime_vetor(float v[])
{
        int i;
        for(i=0;i<10;i++)
        {
            printf("%.2f\t",v[i]);
        }
}
```

Funções

- **Exercício 3:**

```
float acha_maior(float v[])
{       int x;
        float m=v[0];
         for(x=0;x<10;x++)
         {
                if(m<v[x])
                      m=v[x];
         }
         return m;
}
void divide_maior(float v[],float m)
{        int x;
         for(x=0;x<10;x++)
         {
                v[x]/=m;
         }
}
```

Funções

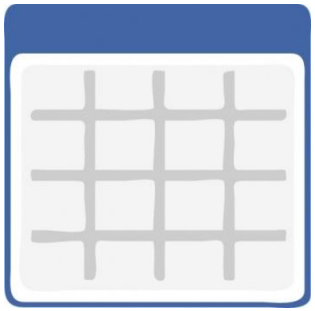
- **Exercício 3:**

C:\Users\Talita\Documents\FACENS\Algoritmos e ProgramaþÒo\Lista 12 - Funþßes\6 MaiorDivide.exe

```
Digite o elemento [0]: 3
Digite o elemento [1]: 5
Digite o elemento [2]: 6
Digite o elemento [3]: 1
Digite o elemento [4]: 8
Digite o elemento [5]: 9
Digite o elemento [6]: 0
Digite o elemento [7]: 10
Digite o elemento [8]: 2
Digite o elemento [9]: 4

Vetor Original:
3.00    5.00    6.00    1.00    8.00    9.00    0.00    10.00   2.00    4.00
Maior = 10.00


Vetor Modificado:
0.30    0.50    0.60    0.10    0.80    0.90    0.00    1.00    0.20    0.40
```

Funções

- **Exercício 4:**
  - Crie uma função capaz de multiplicar uma linha de uma matriz por um dado número. Utilize uma matriz com 4 linhas e 4 colunas. Crie também uma segunda função para ser chamada na main() para mostrar tanto a matriz original quanto a matriz modificada.



Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 4:**

```
#include<stdio.h>
void mostra_matriz(float m[][4]);
void multiplica_matriz(float m[][4], int p,float n);
main()
{     float matriz[4][4],numero;
      int i,j,op,parametro;
      for(i = 0 ; i < 4 ; i++)
      {
          for (j= 0 ; j < 4 ; j++)
          {
             printf("Digite o elemento [%i][%i]: ",i,j);
             scanf("%f", &matriz[i][j]);
          }
      }
      mostra_matriz(matriz);
      printf("Digite a linha e o numero para multiplicar: ");
      scanf("%i %f",&parametro,&numero);
```

Funções

- **Exercício 4:**

```
        multiplica_matriz(matriz,parametro,numero);
        mostra_matriz(matriz);
        getch();
        return 0;
}
void mostra_matriz(float m[][4])
{       int i,j;
        printf("\tCol 0\tCol 1\t Col 2\tCol 3");
        for(i = 0 ; i < 4 ; i++)
        {
                printf("\nLin %i\t", i);
                for (j= 0 ; j < 4 ; j++)
                        printf("%.2f\t", m[i][j]);
                printf("\n");
        }
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercício 4:**

```
void multiplica_matriz(float m[][4],int p, float n)
{
     int i,j;
     for(i = 0 ; i < 4 ; i++)
     {
          for (j= 0 ; j < 4 ; j++)
          {
               if(i==p)
               {
                    m[i][j] = m[i][j]*n;
               }
          }
     }
}
```


Facens
AQUI TEM ENGENHARIA
material didático

Funções

- **Exercícios de funções – chamada por valor**
- **Exercício 5:**
  - Pedir a idade para o usuário e calcular quantos meses e dias de vida ele tem aproximadamente. Utilize uma função para retornar o resultado do processamento.
- **Exercício 6:**
  - Calcular o consumo médio (Cm) de gasolina de um tanque de automóvel. Pedir para o usuário entrar com a distância (km) e volume (litros). Utilize a seguinte fórmula na função de consumo médio: Cm = d (km) / v (litros).
- **Exercício 7:**
  - Crie uma programa que calcula a área de uma esfera através de uma função.

$$A = 4 * \pi * r^2$$

Facens
AQUI TEM ENGENHARIA
material didático